# Constrained Pseudorandom Functions, Revisited

**Julian Mouthon, Mario Razafinony**
Master 1 Cryptologie, Calcul haute-performance et Algorithmique
Sorbonne Université

19 mars 2026

## Introduction

Ce projet s'intéresse aux fonctions pseudo-aléatoires contraintes (*Constrained Pseudorandom Functions*, CPRF), une extension des fonctions pseudo-aléatoires classiques permettant de restreindre les évaluations à un sous-ensemble d'entrées défini par une contrainte.

Dans un premier temps, nous présentons les notions fondamentales liées aux fonctions pseudo-aléatoires et aux CPRFs, ainsi que la construction proposée par Bost, Minaud et Ohrimenko en 2017 (BMO17).

Dans un second temps, nous nous intéressons aux travaux récents de Cheng et Jaeger en 2025 (CJ25), qui mettent en évidence une attaque contre cette CPRF et proposent une variante hachée assurant la sécurité face à un tel attaquant.

*Le code source est disponible sur GitHub : https://github.com/julian-mtn/bmo17-cprf.git*

# Table des matières

**1 Définitions** **4**

Fonction pseudo-aléatoire . . . . . 4

Contrainte . . . . . 4

Fonction pseudo-aléatoire contrainte . . . . . 4

**2 Construction de la CPRF de BMO17** **5**

2.1 Propriétés . . . . . 5

Génération et structure de la clé maîtresse . . . . . 5

Évaluation de la CPRF avec la clé maîtresse . . . . . 5

Dérivation et structure de la clé contrainte . . . . . 5

Évaluation restreinte avec une clé contrainte . . . . . 5

2.2 CPRF avec la permutation à trappe RSA . . . . . 6

2.2.1 Implémentation de la permutation à trappe RSA . . . . . 6

2.2.2 Implémentation de la clé maîtresse et génération de l'état initial . . . . . 6

2.2.3 Implémentation de la dérivation des clés contraintes . . . . . 6

2.3 CPRF avec une permutation linéaire $F^{WEAK}$ . . . . . 7

2.3.1 Implémentation de la permutation . . . . . 7

2.3.2 Implémentation de la clé contraintes . . . . . 7

2.3.3 Approche d'une attaque à base d'un système linéaire . . . . . 7

2.4 Étude d'autres permutations alternatives . . . . . 9

2.4.1 Permutation à trappe de Rabin . . . . . 9

2.4.2 Permutation à trappe AES . . . . . 11

**3 L'attaque CJ25** **11**

3.1 Description générale et intuition de l'attaque . . . . . 11

3.2 Modélisation du jeu de sécurité et avantage de l'attaquant . . . . . 12

3.3 Mise en œuvre expérimentale de l'attaque . . . . . 13

3.3.1 Implémentation de l'oracle de sécurité . . . . . 13

3.3.2 Implémentation de l'attaquant et stratégie . . . . . 13

**4 Renforcement de la CPRF par hachage** **13**

4.1 Principe général de la CPRF hachée . . . . . 13

4.1.1 Implémentation . . . . . 14

4.2 Construction explicite de la CPRF hachée de Cheng et Jaeger . . . . . 14

4.3 Évaluation expérimentale et comparaison des attaques . . . . . 15

**5 Comment lancer le programme** **17**

# 1 Définitions

**Fonction pseudo-aléatoire (PRF)**

Une fonction pseudo-aléatoire

$$F : \mathcal{K} \times D \to R$$

est une fonction telle que, pour une clé secrète $K$, la fonction $F(K, \cdot)$ est indiscernable d'une fonction réellement aléatoire $G : D \to R$ pour tout adversaire efficace.

**Contrainte**

Une contrainte est un prédicat booléen

$$C : D \to \{0, 1\},$$

qui partitionne le domaine $D$ en deux ensembles :

- les entrées *autorisées*, telles que $C(x) = 1$
- les entrées *interdites* (ou contraintes), telles que $C(x) = 0$

Intuitivement, la contrainte spécifie l'ensemble des entrées sur lesquelles une clé contrainte est autorisée à évaluer la fonction pseudo-aléatoire.

Ici, nous considérons principalement des contraintes de la forme

$$C_n(x) = [x < n],$$

qui autorisent toutes les entrées strictement inférieures à un seuil $n \in \mathbb{N}$.

**Fonction pseudo-aléatoire contrainte (CPRF)**

Une fonction pseudo-aléatoire contrainte (CPRF) est donnée par quatre algorithmes efficaces :

- $\mathsf{Setup}(1^\lambda) \to K$ (clé maître) : génère la clé secrète principale $K$.
- $\mathsf{Constrain}(K, C) \to K_C$ (clé contrainte) : produit une clé spéciale $K_C$ permettant d'évaluer la fonction uniquement sur les entrées autorisées par $C$.
- $\mathsf{Eval}(K, x) \to y$ : évalue la fonction pseudo-aléatoire sur une entrée $x$ avec la clé maître.
- $\mathsf{EvalC}(K_C, x) \to y$ : évalue la fonction sur l'entrée $x$ avec la clé contrainte $K_C$.

Une CPRF est correcte si, pour toute clé $K$, toute contrainte $C$ et toute entrée $x$ telle que $C(x) = 1$, on a

$$\mathsf{EvalC}(K_C, x) = \mathsf{Eval}(K, x).$$

Autrement dit, la clé contrainte permet d'évaluer la PRF exactement sur les entrées autorisées.

# 2 Construction de la CPRF de BMO17

## 2.1 Propriétés

La CPRF de BMO17 est basée sur l'itération successive d'une permutation à trappe, c'est-à-dire une fonction bijective facile à calculer mais difficile à inverser sans information secrète. On note cette permutation $\pi$.

**Génération de la clé maîtresse**

La clé maîtresse est composée des éléments suivants :

- $ST_0 \in \mathbb{Z}_N$, un état initial secret choisi aléatoirement
- $SK$, une clé secrète RSA définissant la permutation à trappe $\pi_{SK}$

La clé publique associée $PK$ permet uniquement de calculer la permutation directe $\pi$.

**Evaluation de la CPRF avec la clé maîtresse**

Avec la clé maîtresse $(ST_0, SK)$ et une entrée $c$, on peut évaluer la CPRF sur $c$ par :

$$Eval((SK, ST_0), c) = \pi_{SK}^{-c}(ST_0)$$

L'évaluation consiste à appliquer $c$ fois l'inverse de la permutation à partir de l'état initial $ST_0$.

$$\boxed{ST_0} \xrightarrow{\pi^{-1}} \boxed{ST_1} \xrightarrow{\pi^{-1}} \cdots \xrightarrow{\pi^{-1}} \boxed{ST_n}$$

**Génération de la clé contrainte**

À partir de la clé maîtresse $(ST_0, SK)$ et d'un entier $n$, correspondant à la contrainte $C(c) = [c < n]$, la clé contrainte est composée des éléments suivants :

- $PK$, la clé publique associée à la permutation $\pi$
- $ST_n = \pi_{SK}^{-n}(ST_0)$
- $n$

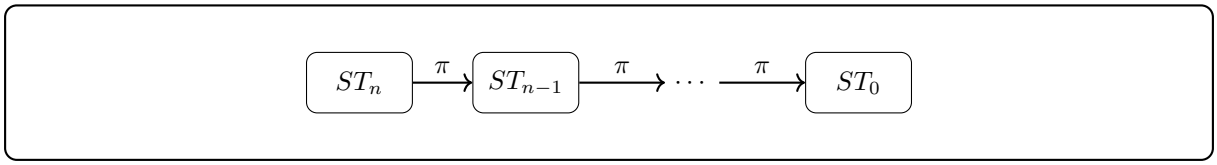
La clé secrète $SK$ n'est pas incluse dans la clé contrainte.

**Évaluation de la CPRF avec la clé contrainte**

Avec la clé contrainte $(PK, ST_n, n)$ et une entrée $c$, l'évaluation est possible uniquement si $c < n$. Dans ce cas, la valeur de la CPRF est calculée comme suit :

$$\mathsf{EvalC}((PK, ST_n, n), c) = \pi_{PK}^{n-c}(ST_n)$$

Cette valeur est égale à $Eval((SK, ST_0), c)$ par construction, ce qui assure la correction du schéma.




## 2.2 CPRF avec la permutation à trappe RSA

Nous utilisons la bibliothèque OpenSSL pour la gestion des grands entiers (`BIGNUM`), les opérations modulaires et le hachage SHA-256.

L'aléa est sécurisé et est obtenu à partir de `/dev/urandom`, ce qui permet de générer l'état initial $ST_0$ ainsi que les paramètres RSA de manière plus sûre.

### 2.2.1 Implémentation de la permutation à trappe RSA

La permutation à trappe utilisée dans BMO17 est défini à l'aide de RSA. Nous implémentons la génération de clés RSA de taille 4096 bits, ainsi que l'évaluation de la permutation dans les deux sens.

Plus précisément, la fonction publique correspond à :

$$x \mapsto x^e \bmod N,$$

tandis que l'inverse de la permutation est calculé via :

$$x \mapsto x^d \bmod N.$$

avec

$$x^{e \cdot d} = x \bmod N, \forall x \in \mathbb{Z}_N.$$

Afin de limiter les fuites par canaux auxiliaires, l'évaluation privée est réalisée à l'aide de la fonction `BN_mod_exp_mont_consttime` d'OpenSSL, qui garantit un temps d'exécution indépendant des données.

### 2.2.2 Implémentation de la clé maîtresse et génération de l'état initial

La clé maîtresse de la CPRF est composée d'un état initial secret $ST_0$ et d'une clé privée RSA. L'état initial est généré comme un entier aléatoire de 256 bits à partir de `/dev/urandom`..

L'évaluation avec la clé maîtresse consiste à appliquer $c$ fois l'inverse de la permutation RSA à partir de $ST_0$.

### 2.2.3 Implémentation de la dérivation des clés contraintes

À partir de la clé maîtresse et d'un paramètre de contrainte $n$, nous dérivons une clé contrainte composée de la clé publique RSA $(e, N)$ et de l'état

$$ST_n = \pi^{-n}(ST_0).$$

Cette clé permet ensuite d'évaluer la CPRF uniquement pour les entrées $c < n$ en appliquant $(n - c)$ fois la permutation publique à partir de $ST_n$. La clé privée n'est jamais incluse dans la clé contrainte, ce qui garantit que seules les évaluations autorisées sont possibles.

## 2.3 CPRF avec une permutation linéaire $F^{WEAK}$

Dans cette section, nous allons voir une implémentation d'une CPRF dont l'entrée et la sortie seront des vecteurs $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ et $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^M$ où $p$ est un nombre premier et $N$ et $M$ les dimensions des vecteurs.

### 2.3.1 Implémentation de la permutation

Ici, l'évaluation de la permutation à trappe est définie par :

$$Eval(F^{WEAK}, \vec{x}) = Eval(S, \vec{x}) = S \cdot \vec{x}$$

où $S$ est une matrice de taille $M \times N$ à coefficients dans $\mathbb{Z}/p\mathbb{Z}$ où $M << N$.

$S$ est la clé maîtresse et elle est choisie aléatoirement lors de la génération de la clé maîtresse.

### 2.3.2 Implémentation de la clé contraintes

Pour générer la clé contrainte :

- l'attaquant envoie un vecteur $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^N$
- l'oracle tire au hasard un vecteur $\vec{d} \in (\mathbb{Z}/p\mathbb{Z})^M$ tel que $\vec{(}d) \neq \vec{(}0)$
- l'oracle renvoie $Constrain(S, \vec{y}) = S_{\vec{y}} = S + d \cdot \vec{y}^T$

En posant

$$CEval(S_{\vec{y}}, \vec{x}) = S_{\vec{y}} \cdot \vec{x}$$

on remarque que l'ensemble des vecteurs contraints sont les $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ tel que $< \vec{x}, \vec{y} >= 0$.

En effet,

$$CEval(S_{\vec{y}}, \vec{x}) = S_{\vec{y}} \cdot \vec{x}$$

$$CEval(S_{\vec{y}}, \vec{x}) = S \cdot \vec{x} + d \cdot \vec{y}^T \cdot \vec{x}$$

$$CEval(S_{\vec{y}}, \vec{x}) = S \cdot \vec{x} + d \cdot < \vec{x}, \vec{y} >$$

$$CEval(S_{\vec{y}}, \vec{x}) = S \cdot \vec{x} \iff < \vec{x}, \vec{y} >= 0$$

Donc :

$$S_{\vec{y}} \cdot \vec{x} = \begin{cases} S \cdot \vec{x} & \text{si } \langle \vec{x}, \vec{y} \rangle = 0 \\ \text{résultat 'brouillé'} & \text{sinon} \end{cases}$$

Cette construction permet donc de restreindre l'évaluation de la fonction aux seuls vecteurs orthogonaux à $\vec{y}$.

### 2.3.3 Approche d'une attaque à base d'un système linéaire

Notons $\vec{S}i \in (\mathbb{Z}/p\mathbb{Z})^N$ la ième ligne de $S$.

Pour un $\vec{x}$ orthogonal à $\vec{y}$, pour chaque $\vec{S}i$, notons $k_i = \vec{S}i \cdot \vec{x} = \vec{S}i_{\vec{y}} \cdot \vec{x}$.

$$\begin{array}{ccccc} S & & \vec{x} & & \vec{k} \\ \begin{bmatrix} \vec{S_1} \\ \vec{S_2} \\ \vdots \\ \vec{S_M} \end{bmatrix} & \cdot & \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} & = & \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_M \end{bmatrix} \end{array}$$

FIGURE 1 – Représentation du système linéaire $S \cdot \vec{x} = \vec{k}$

En choisissant $N$ vecteurs $\{\vec{x_j}\}_{1 \leq j \leq N}$ tel que $< \vec{x_j}, \vec{y} >= 0, \forall j$, on peut construire la matrice $X \in (\mathbb{Z}/p\mathbb{Z})^{N \times N}$ tel que la jème ligne de $X$ soit $\vec{x_j}^T$

$$\begin{bmatrix} \vec{x_1} \\ \vec{x_2} \\ \vdots \\ \vec{x_N} \end{bmatrix}$$

FIGURE 2 – Matrice $X$ pour l'attaque construite avec $\{\vec{x_j}\}$ orthogonaux à $\vec{y}$

Pour retrouver la matrice $S$, il suffit donc de résoudre le système linéaire $X \cdot \vec{Si}^T = \vec{k_i}$ où le jème élément de $\vec{k_i}$ est $k_{i_j} = \vec{Si} \cdot \vec{x_j}$, pour chaque ligne $S_i$ de $S$.

$$\begin{array}{ccccc} X & & \vec{S_i}^T & & \vec{k_i} \\ \underbrace{\begin{bmatrix} \vec{x_1} \\ \vec{x_2} \\ \vdots \\ \vec{x_N} \end{bmatrix}}_{N \times N} & \cdot & \underbrace{\begin{bmatrix} S_{i,1} \\ S_{i,2} \\ \vdots \\ S_{i,N} \end{bmatrix}}_{N \times 1} & = & \underbrace{\begin{bmatrix} k_{i_1} \\ k_{i_2} \\ \vdots \\ k_{i_N} \end{bmatrix}}_{N \times 1} \end{array}$$

FIGURE 3 – Représentation du système linéaire $X \cdot \vec{S_i}^T = \vec{k_i}$, où $k_{i_j} = \vec{S_i} \cdot \vec{x_j}$

On répète $M$ fois ce procédé pour retrouver chacun des $\vec{Si}$.

**Problème :** Le système linéaire admet une solution unique si et seulement si la matrice $X$ est inversible, ce qui est équivalent à $\text{rang}(X) = N$. Afin de garantir cette inversibilité, il est nécessaire de choisir des vecteurs $\vec{x_j}$ tels que la famille $\{\vec{x_j}\}_{1 \leq j \leq N}$ soit libre, c'est-à-dire qu'elle engendre un espace vectoriel de dimension $N$.

Par ailleurs, chaque vecteur $\vec{x_j}$ doit être orthogonal à $\vec{y}$. On a alors que l'ensemble $\{\vec{y}\} \cup \{\vec{x_j}\}_{1 \leq j \leq N}$ est constitué de $N + 1$ vecteurs deux à deux linéairement indépendants, et engendrerait donc un espace de dimension $N + 1$.

Ceci est impossible, puisque ces vecteurs appartiennent tous à $(\mathbb{Z}/p\mathbb{Z})^N$, qui est de dimension $N$. Cette contradiction montre qu'il est impossible de construire une telle famille de vecteurs $\{\vec{x_j}\}_{1\leq j\leq N}$ satisfaisant simultanément ces conditions.

**Alternative :** On remarque tout de même que $\vec{y}$ engendre un espace de dimension 1, ce qui implique qu'il est possible de choisir $N-1$ vecteurs $\{\vec{x_j}\}_{1\leq j\leq N-1}$ tel que chaque vecteur $\vec{x_j}$ soit orthogonal à $\vec{y}$.

On peut donc reconstruire une autre matrice $X$ de taille $(N-1)\times(N-1)$ puis poser le système linéaire suivant :

$$\begin{array}{ccccc} X & & \vec{S_i}^T & & \vec{k_i} \\ \underbrace{\begin{bmatrix} x_{1,1} & \cdots & x_{1,N-1} \\ x_{2,1} & \cdots & x_{2,N-1} \\ \vdots & \vdots & \vdots \\ x_{N-1,1} & \cdots & x_{N-1,N-1} \end{bmatrix}}_{(N-1)\times(N-1)} & \cdot & \underbrace{\begin{bmatrix} S_{i,1} \\ S_{i,2} \\ \vdots \\ S_{i,N-1} \end{bmatrix}}_{(N-1)\times 1} & = & \underbrace{\begin{bmatrix} k_{i_1} - x_{1,N}\cdot S_{i,N} \\ k_{i_2} - x_{2,N}\cdot S_{i,N} \\ \vdots \\ k_{i_{N-1}} - x_{N-1,N}\cdot S_{i,N} \end{bmatrix}}_{(N-1)\times 1} \end{array}$$

FIGURE 4 – Représentation du nouveau système linéaire $X\cdot\vec{S_i}^T = \vec{k_i}$, où $k_{i_j} = \vec{S_i}\cdot\vec{x_j}$

Ici, l'ensemble des vecteurs $\{\vec{x_j}\}_{1\leq j\leq N-1}$ forme une famille libre, donc $rang(X) = N-1$, donc $X$ est inversible, donc ce système linéaire possède une unique solution. Cependant, cette unique solution est calculée en fonction de $S_{i,N}$.

En répétant ce procédé $M$ fois, on peut donc réécrire chaque ligne $S_i$ de $S$ en fonction d'un élément $S_{i,N}$.

**Remarque :** Le choix d'utiliser un élément $S_{i,N}$ pour écrire chaque ligne $S_i$ de $S$ est arbitraire. On aurait pu réécrire chaque ligne $S_i$ de $S$ avec n'importe quel élément $S_{i,l}$. Dans ce cas, dans le système linéaire précédent, chaque ligne $\vec{x_i}$ de $X$ sera $(x_{i,1},\ldots,x_{i,l-1},x_{i,l+1},\ldots,x_{i,N}), \forall i\neq l$, le vecteur $\vec{S_i}^T$ sera $(S_{i,1},\ldots,S_{i,l-1},S_{i,l+1},\ldots,S_{i,N})$ et chaque élément de $\vec{k_i}$ sera $k_{i_j} - x_{j,l}\cdot S_{i,l}, \forall i\neq l$

TODO : Trouver une attaque qui permet de distingue une fonction aléatoire de l'application de $S$ à un vecteur à partir de cette reconstruction de $S$.

## 2.4 Étude d'autres permutations alternatives

### 2.4.1 Permutation à trappe de Rabin

La permutation de Rabin est une fonction cryptographique à trappe fondée sur la difficulté de la factorisation des entiers. Elle est conceptuellement plus simple que RSA et repose sur l'opération de mise au carré modulo un entier composé.

— **Génération de la clé :**

On choisit deux nombres premiers distincts $p$ et $q$ tels que :

$$p \equiv q \equiv 3 \pmod 4$$

La clé publique est définie par :

$$n = p \cdot q$$

La clé privée est le couple $(p, q)$.

Cette condition sur $p$ et $q$ permet un calcul efficace des racines carrées modulo ces nombres premiers.

— **Permutation de Rabin :**

La permutation de Rabin est la fonction :

$$f : \mathbb{Z}_n \to \mathbb{Z}_n$$

définie par :

$$f(x) = x^2 \bmod n$$

Cette fonction est facile à calculer, mais difficile à inverser sans connaître la factorisation de $n$.

— **Inverser la permutation de Rabin :**

Inverser la permutation de Rabin revient à résoudre l'équation :

$$x^2 \equiv y \pmod{n}$$

Lorsque $n = p \cdot q$, cette équation admet exactement **quatre solutions distinctes** modulo $n$.

On commence par réduire le problème modulo $p$ et $q$ :

$$\begin{cases} x^2 \equiv y \pmod{p} \\ x^2 \equiv y \pmod{q} \end{cases}$$

Comme $p \equiv 3 \pmod{4}$, une racine carrée modulo $p$ est donnée par :

$$r_p = y^{\frac{p+1}{4}} \bmod p$$

De même, modulo $q$ :

$$r_q = y^{\frac{q+1}{4}} \bmod q$$

Les solutions sont donc :

— $X_0 = (r_p * q * q^{-1}_{\bmod p} + r_q * p * p^{-1}_{\bmod q}) \bmod n$
— $X_1 = (r_p * q * q^{-1}_{\bmod p} - r_q * p * p^{-1}_{\bmod q}) \bmod n$
— $X_2 = (-r_p * q * q^{-1}_{\bmod p} + r_q * p * p^{-1}_{\bmod q}) \bmod n$
— $X_3 = (-r_p * q * q^{-1}_{\bmod p} - r_q * p * p^{-1}_{\bmod q}) \bmod n$

**Problème :** Laquelle de ces racines correspond au message original ?

La fonction $f(x) = x^2 \bmod n$ n'est pas injective : chaque chiffré $c$ a 4 pré-images. Même avec la clé secrète $(p, q)$, on peut calculer toutes les racines mais **pas déterminer directement celle qui correspond au message initial**. Appliquer l'inverse $n - c$ fois n'assure pas de retrouver le message original.

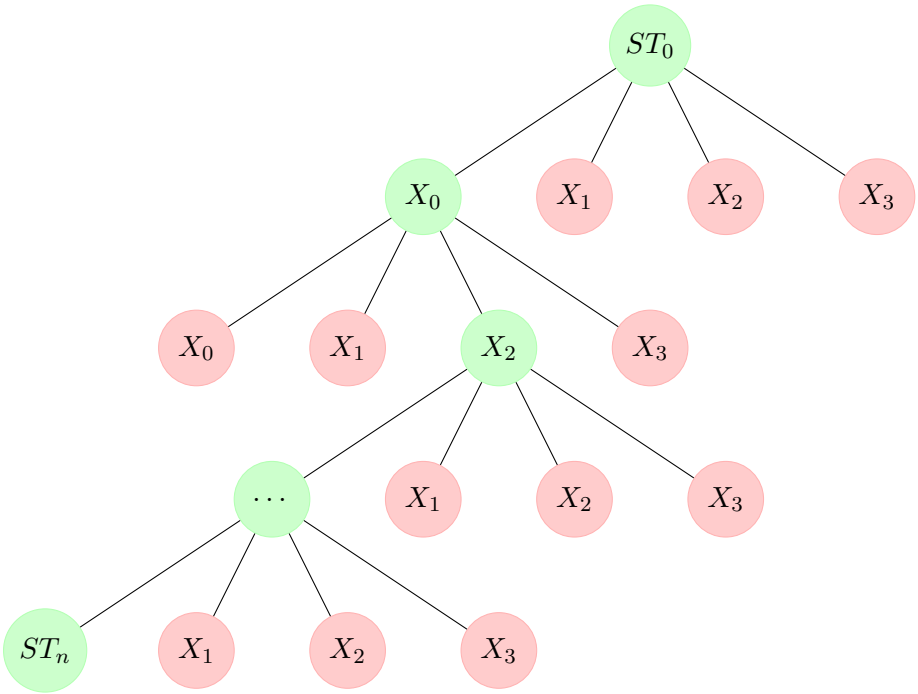



FIGURE 5 – Arbre des pré-images dans la permutation de Rabin.

**Conclusion :** La permutation de Rabin n'est pas adaptée pour notre CPRF car chaque valeur a quatre pré-images possibles. Même avec la clé privée, il est impossible de savoir laquelle correspond à l'entrée originale, ce qui empêche l'application répétée de l'inverse $(n-c)$ fois pour retrouver correctement le résultat de la CPRF.

### 2.4.2 Permutation à trappe AES

Le chiffrement AES applique une fonction bijective $E_k(x)$ qui dépend de la clé $k$. Contrairement à $x \mapsto x^2 \bmod n$, chaque bloc chiffré a une **unique pré-image** pour une clé donnée. Cependant, pour déchiffrer, **il faut absolument connaître la clé k** :

— Sans la clé, l'attaquant ne peut pas inverser la permutation.

— AES reste sécurisé grâce à la clé secrète.

Donc l'utilisation d'AES comme permutation à trappe n'est pas adaptée pour notre CPRF.

# 3 L'attaque CJ25

## 3.1 Description générale et intuition de l'attaque

L'attaque CJ25 exploite le fait qu'un attaquant peut obtenir une clé contrainte permettant d'évaluer la fonction pseudo-aléatoire sur un sous-ensemble d'entrées, tout en conservant l'accès à un oracle d'évaluation global.

Dans un premier temps, l'adversaire choisit un indice $n$ et demande à l'oracle une clé contrainte $(PK, ST_n, n)$. Cette clé lui permet d'évaluer la fonction sur toute entrée $x$ telle que $x > n$ à l'aide des permutations publiques associées.

Dans un second temps, l'adversaire interroge l'oracle d'évaluation sur une entrée $x > n$ et obtient une valeur $ST_x$. À ce stade, l'attaquant ne sait pas si $ST_x$ provient d'une fonction pseudo-aléatoire contrainte ou d'une fonction aléatoire.

À l'aide de la clé contrainte, l'attaquant applique successivement les permutations publiques afin de calculer la valeur $\pi_{PK}^{x-n}(ST_x)$. Si l'attaquant retrouve $ST_a = ST_x$, il peut alors conclure que la valeur $ST_x$ est le résultat d'une fonction pseudo-aléatoire contrainte.

Il est donc possible pour un attaquant d'avoir des informations sur la structure de la CPRF à partir de la clé contrainte et des évaluations obtenues, ce qui lui permet de distinguer une CPRF d'une fonction aléatoire avec une probabilité non négligeable.

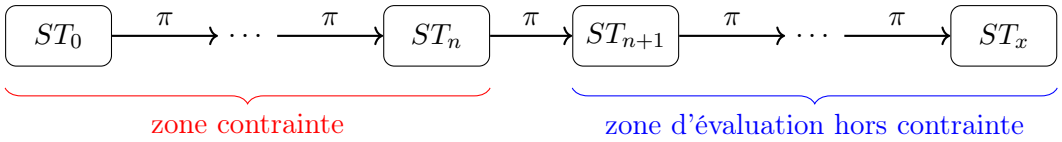



FIGURE 6 – Schéma général de l'attaque CJ25

## 3.2 Modélisation du jeu de sécurité et avantage de l'attaquant

**Jeu de sécurité pour les CPRF**

L'attaquant $\mathcal{A}$ interagit avec un oracle $O$ qui est soit une $CPRF$ soit une fonction aléatoire. L'objectif de l'attaquant est de distinguer les deux cas.

- **Phase 1 :** $\mathcal{A}$ demande une clé contrainte $K_C = (PK, ST_n, n)$ pour une contrainte $n$ de son choix, lui permettant d'évaluer la fonction sur un sous-ensemble d'entrées.
- **Phase 2 :** $\mathcal{A}$ fait un nombre de requêtes d'évaluation à l'oracle $O$ sur des entrées $x > n$ pour obtenir des valeurs $ST_x$ non contraintes.
- **Phase 3 :** $\mathcal{A}$ utilise les valeurs obtenues pour évaluer $ST_a = \pi_{PK}^{n-x}(ST_x)$ et compare avec $ST_n$.
- **Décision :** $\mathcal{A}$ décide que $O$ est la $CPRF$ si $ST_a = ST_n$, sinon il décide que $O$ est une fonction aléatoire.

Dans le cas où $O$ est une $CPRF$, l'attaquant $\mathcal{A}$ retrouve toujours $ST_a = ST_n$, et il ne se trompe jamais. En revanche, si $O$ est une fonction aléatoire, la probabilité que $ST_a = ST_n$ est de $1/N$, où $N$ est la taille de l'espace de sortie de la fonction. Dans ce cas, l'attaquant se trompe avec une probabilité de $1/N$. L'attaquant $\mathcal{A}$ gagne donc avec une probabilité de $1 - 1/N$ de distinguer correctement les deux cas, ce qui est non négligeable pour des tailles de $N$ raisonnables.

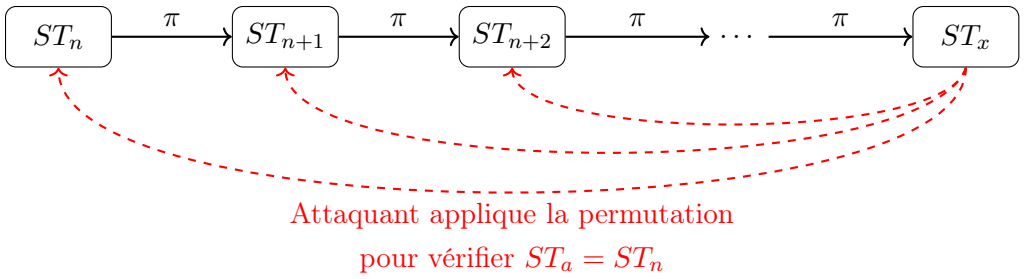



FIGURE 7 – Schéma des permutation de l'attaque dans la zone hors contrainte

### 3.3 Mise en œuvre expérimentale de l'attaque

Pour simuler le jeu de sécurité, nous avons mis en place une architecture client–serveur locale reposant sur TCP. Le serveur implémente l'oracle du jeu (évaluation et génération de clés contraintes), le client joue le rôle de l'attaquant (requêtes d'évaluation et de contrainte, tests, ...).

#### 3.3.1 Implémentation de l'oracle de sécurité

L'oracle met à disposition deux interfaces accessibles à l'attaquant :

— `EVAL(x)` : retourne une valeur associée à l'entrée $x$.
En monde PRF, l'oracle retourne l'évaluation de la CPRF à l'aide de la clé maîtresse.
En monde aléatoire, il retourne une valeur uniforme de même taille.

— `CONSTRAIN(n)` : retourne une clé contrainte associée à l'indice $n$, permettant l'évaluation de la fonction sur un sous-ensemble d'entrées.

#### 3.3.2 Implémentation de l'attaquant et stratégie

L'attaquant choisit un indice $n$ et demande à l'oracle la clé contrainte correspondante. Cette clé lui permet d'évaluer la fonction sur des entrées strictement supérieures à $n$.

Ensuite il effectue des requêtes `EVAL` sur des entrées $x > n$. À partir des valeurs retournées par l'oracle et de la clé contrainte, il applique les permutations publiques afin de tester la cohérence de la structure de la CPRF.

Si l'attaquant trouve une correspondance, il peut conclure que l'oracle implémente une CPRF plutôt qu'une fonction aléatoire.

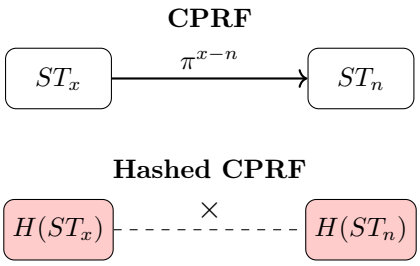
## 4 Renforcement de la CPRF par hachage

Afin d'améliorer la sécurité de la construction de BMO17 face à l'attaque CJ25, il est possible d'implémenter une version hachée de la CPRF.

### 4.1 Principe général de la CPRF hachée

L'idée est d'introduire une fonction de hachage cryptographique dans l'évaluation. L'attaquant pourra toujours évaluer la CPRF sur les entrées non contraintes, mais il ne pourra pas exploiter les résultats pour distinguer la CPRF d'une fonction aléatoire car ceux-ci seront hachés de manière irréversible.

**→ Le rôle de la fonction de hachage est de rendre impossible toute utilisation ou comparaison via les permutations publiques.**




### 4.1.1 Implémentation

Nous avons implémenté deux versions de la CPRF hachée :

- **CPRF hachée simple** : l'évaluation est effectuée normalement, puis le résultat est haché à l'aide de `SHA-256` avant d'être retourné.
  - $Eval_H(x)$ : retourne le hachage de l'évaluation sur l'entrée $x$ avec la clé maîtresse, c'est-à-dire $H(Eval((SK, ST_0), x))$.
  - $EvalC_H(n)$ : retourne le hachage de l'évaluation sur l'entrée $n$ avec la clé contrainte, c'est-à-dire $H(EvalC((PK, ST_n, n), c))$.
- **CPRF hachée avec lazy sampling** : afin de se rapprocher du modèle théorique d'un oracle de hachage idéal utilisé dans l'article CJ25, nous avons également implémenté une version reposant sur le lazy sampling décrite dans la section suivante.

## 4.2 Construction explicite de la CPRF hachée de Cheng et Jaeger

Nous présentons ici la version hachée proposée dans l'article CJ25.

Notations utilisées :

- $P$ : fonction de hachage idéale modélisée comme un oracle de hachage aléatoire.
- $\sigma_P$ : état interne de $P$ (initialement vide) qui mémorise les paires $(x, y)$.
- $\lambda$ : paramètre de sécurité (taille de la sortie de $P$).
- $k$ : clé secrète de la $\mathsf{CPRF}$.
- $k_R$ : clé contrainte associée à un ensemble de restrictions $R$.
- $x$ : entrée de la fonction.
- $y \in \{0,1\}^\lambda$ : valeur de hachage associée à une entrée.

L'évaluation est définie par :

$$\begin{aligned}
\mathsf{Hashed[CPRF].Eval}(1^\lambda, k, x) &= P.\mathsf{LazySampling}(1^\lambda, \mathsf{CPRF.Eval}(1^\lambda, k, x) : \sigma_P). \\
\mathsf{Hashed[CPRF].EvalC}(1^\lambda, k_R, R, x) &= P.\mathsf{LazySampling}(1^\lambda, \mathsf{CPRF.EvalC}(1^\lambda, k_R, R, x) : \sigma_P).
\end{aligned}$$

**Lazy Sampling**

Chercher $x$ dans $\sigma_P$
Si $x \notin \sigma_P$ :
  Choisir $y \leftarrow \{0,1\}^\lambda$ au hasard
  Mémoriser $(x, y)$
Sinon :
  Récupérer $y$ associé à $x$
Retourner $y$

*Algorithme de lazy sampling d'une primitive idéale.*

## 4.3 Évaluation expérimentale et comparaison des attaques

Après avoir implémenté les différentes versions de la CPRF (normale, hachée avec SHA-256 et hachée avec lazy sampling), nous avons mené des expériences pour évaluer l'efficacité de l'attaque CJ25 contre chacune d'elles. Les tests ont été réalisés en exécutant 100 requêtes d'évaluation pour chaque version, et en enregistrant le taux de succès de l'attaque à chaque étape.

Dans la figure 8, on constate les résultats sur les attaques sur la version non hachée de la CPRF. On observe que l'attaque CJ25 est très efficace contre la CPRF non hachée, avec une probabilité de succès proche de 1 dès les premières requêtes d'évaluation. Avec $N$ de l'ordre de $2^{4096}$, la probabilité de se tromper est de $1/N$, ce qui est pratiquement nul. Le taux d'attaque réussie à 100% s'explique par le fait que l'attaquant doit faire en espérance $N = 2^{4096}$ requêtes pour se tromper une fois.

Sur les figures 9 et 10, on a utilisé le même modèle d'attaque contre les versions hachées avec SHA-256 et avec le lazy sampling. On observe que l'attaque est complètement inefficace, avec un taux de succès proche de 0.5, ce qui correspond à une attaque aléatoire car l'attaquant ne détecte jamais de CPRF et se trompe dans la moitié des cas. Ces deux figures montrent aussi que le choix du hachage n'a pas d'impact significatif sur l'efficacité de l'attaque, ce qui confirme que l'introduction du hachage rend la structure de la CPRF indiscernable pour l'attaquant.

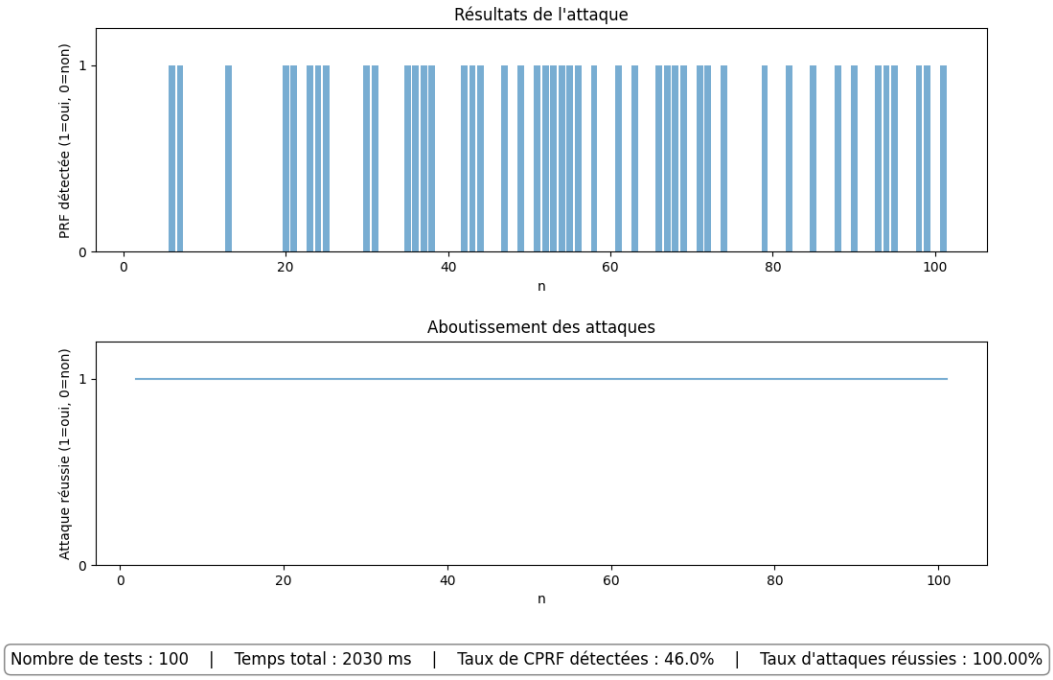



FIGURE 8 – Attaque version non hachée

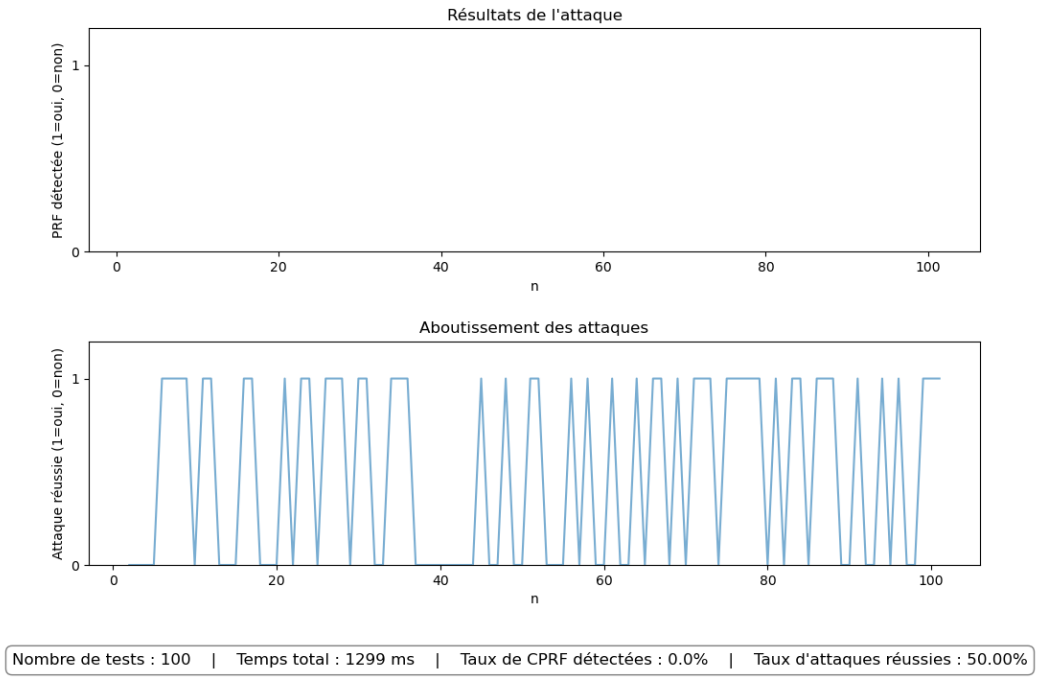



FIGURE 9 – Attaque version hachée avec SHA-256

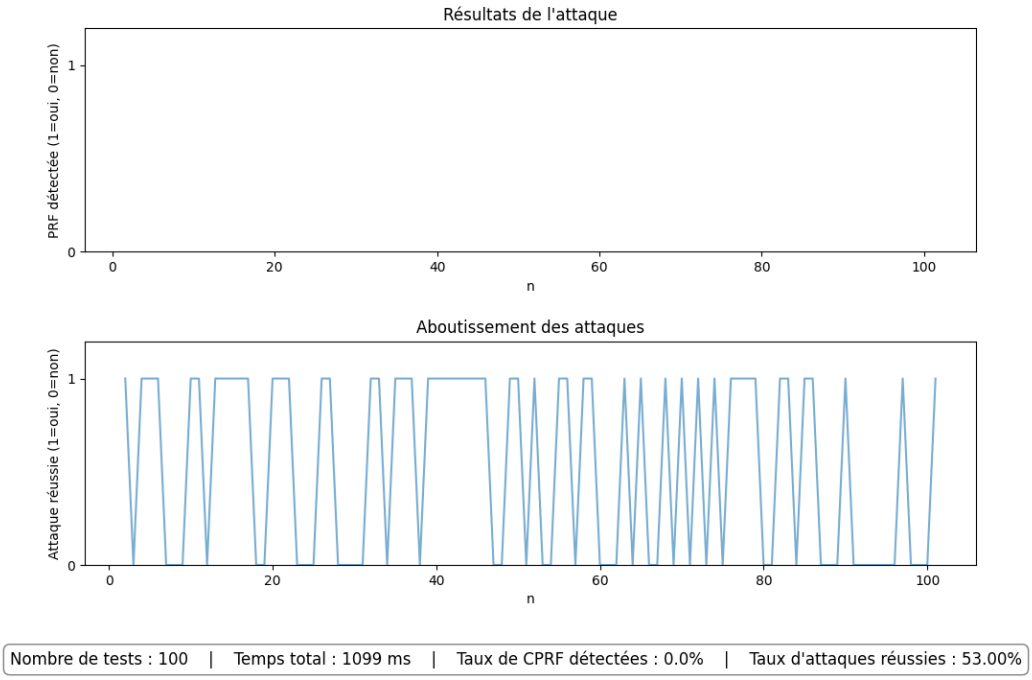



FIGURE 10 – Attaque version hachée avec lazy sampling

# 5 Comment lancer le programme

| **Commande** | **Description** |
|---|---|
| `make` | Compile le projet. |
| `./start.sh -n <size>` | Lance les tests avec la CPRF normale. |
| `./start.sh -h <size>` | Lance les tests avec la CPRF hachée. |
| `./start.sh -l <size>` | Lance les tests avec le lazy sampling. |
| `<size>` | Nombre de tests à exécuter. |
| `python3 display.py` | Trace les courbes à partir des résultats enregistrés dans `attack_results.txt`. |